arXiv:2506.04055v1 [physics.comp-ph] 4 Jun 2025

`chemtrain-deploy`: A parallel and scalable framework for machine learning potentials in million-atom MD simulations.

Paul Fuchs[1], Weilong Chen[1], Stephan Thaler[3], Julija Zavadlav[1,2*]

[1]Professorship of Multiscale Modeling of Fluid Materials, Department of Engineering Physics and Computation, TUM School of Engineering and Design, Technical University of Munich, Germany.
[2]Atomistic Modeling Center (AMC), Munich Data Science Institute (MDSI), Technical University of Munich, Germany.
[3]Valence Labs, Montreal, QC, Canada.

*Corresponding author(s). E-mail(s): julija.zavadlav@tum.de;

Abstract

Machine learning potentials (MLPs) have advanced rapidly and show great promise to transform molecular dynamics (MD) simulations. However, most existing software tools are tied to specific MLP architectures, lack integration with standard MD packages, or are not parallelizable across GPUs. To address these challenges, we present `chemtrain-deploy`, a framework that enables model-agnostic deployment of MLPs in LAMMPS. `chemtrain-deploy` supports any JAX-defined semi-local potential, allowing users to exploit the functionality of LAMMPS and perform large-scale MLP-based MD simulations on multiple GPUs. It achieves state-of-the-art efficiency and scales to systems containing millions of atoms. We validate its performance and scalability using graph neural network architectures, including MACE, Allegro, and PaiNN, applied to a variety of systems, such as liquid–vapor interfaces, crystalline materials, and solvated peptides. Our results highlight the practical utility of `chemtrain-deploy` for real-world, high-performance simulations and provide guidance for MLP architecture selection and future design.

1 Introduction

In recent years, machine learning potentials (MLPs) have advanced rapidly and found widespread applications in fields such as computational chemistry and materials science [1–4]. By training the models on high-accuracy reference datasets, typically consisting of energies and forces, MLPs offer a promising compromise between accuracy and computational efficiency. They approach the accuracy of ab initio methods [5] while maintaining computational speeds closer to classical force fields [6–8]. This performance is achieved by capturing high-order, many-body interactions through either predefined [9, 10] or learned representations of local atomic environments [11, 12], enabling near-linear scaling with system size [13], and making them particularly attractive for large-scale simulations.

While there have been significant advancements [14–16], several challenges still hinder the widespread adoption of MLPs in real-world applications. From a model architecture perspective, graph neural networks (GNNs) have emerged as a powerful approach due to their natural ability to encode atomic topologies and interactions [15, 17–22]. Numerous GNN architectures leveraging geometric priors have been proposed, with many accompanied by specialized software packages such as SchNetPack [23], TorchANI [24], MACE [22], TorchMD [25] and DistMLIP [26]. However, these packages are often tightly coupled to their respective models and typically lack the modularity or plugin interfaces needed for seamless integration with widely used molecular dynamics (MD) software [27–29]. This limits their extensibility and practical usability in domain-specific workflows.

Moreover, the rapid pace of innovation in other aspects of the field presents additional challenges. New data curation strategies [30–32], training methodologies [33–35], and schemes for incorporating long-range interactions [36–40] are being developed continuously. Integrating these advancements into existing software frameworks often demands additional engineering effort, which can lead to a more diverse and specialized ecosystem. This fragmentation makes it harder for practitioners to adopt state-of-the-art methods and for developers to maintain robust, general-purpose tools.

Another emerging concern is how MLP performance is evaluated. While most efforts have focused on minimizing force and energy prediction errors, recent studies argue that simulation stability and the accuracy of observable quantities are more critical for practical applications [41–45]. Benchmarking different architectures against specific simulation tasks is thus gaining importance, as it provides insights that are more relevant for real-world usage and future model development.

In response to these challenges, several projects have attempted to provide easy-to-use interfaces between the ML core and different types of traditional modelling software. Plugins have been developed such as Allegro-LAMMPS [18], SevenNet [46], GROMACS-NNpot [28], FitSNAP [47], and OpenMM-Torch [48] to enable simulations with MD software such as LAMMPS [29], GROMACS [28], and OpenMM [48]. However, many remain constrained by architecture-specific designs or face scalability challenges. DeepMD-kit [49] has recently introduced a multi-backend framework and support for external models [50], but its performance and scalability across multi-GPU systems have yet to be validated.

In this work, we present `chemtrain-deploy`, a model-agnostic deployment framework that extends our exsisting JAX-based training platform, `chemtrain` [51]. `chemtrain` was originally designed to support customizable training of neural network potentials with different training strategies, which integrates with JAX, M.D [52] to offer a unified training and simulation environment. However, current JAX, M.D. lack the robustness, interoperability, and scalability of established packages such as LAMMPS. `chemtrain-deploy` bridges this gap by enabling seamless deployment of pretrained semi-local MLPs into LAMMPS, allowing efficient large-scale simulations with systems containing millions of atoms across mutliple GPUs. To demonstrate flexibility and scalability, we benchmark `chemtrain-deploy` on three widely used GNN models: MACE, Allegro, and PaiNN, all trained to comparable accuracy. We test them on diverse systems including water–vapor coexistence, solid state (fcc) aluminum, and solvated Chignolin, evaluating strong and weak scaling, parallel efficiency, and performance relative to other frameworks such as JAX, M.D. Our results demonstrate the practicality of `chemtrain-deploy` for real-world, high-performance molecular simulation and provide guidance on GNN selection and future development.

2 Results

Structure of `chemtrain-deploy`

`chemtrain-deploy` complements the `chemtrain` [51] framework to apply trained models in large-scale MD simulations using established MD software. Therefore, `chemtrain-deploy` comprises three parts: exporting a trained model, importing it into established MD software through a plugin or modification, and efficiently evaluating the model on high-performance hardware. The parts and workflow of `chemtrain-deploy` are depicted in Figure 1.

First, `chemtrain-deploy` extends the framework `chemtrain` (Fig. 1a) to export trained models to a self-contained format (Fig. 1b). The export saves the model architecture and parameters through the MLIR framework [53] and properties that define the input and output of the model, such as length and energy units, the maximum length of graph edges, and the format of the input graph. Therefore, the exported model file contains all the information needed to apply the model and is thus simple to share or archive within the JAX compatibility guarantees.

Secondly, `chemtrain-deploy` consists of a plugin to load and use the exported model for large-scale molecular dynamics simulations in established MD software (Fig. 1c). The MD software provides basic and advanced algorithms to perform MD simulations. Moreover, the MD software provides algorithms to decompose the system into multiple domains for parallelization and create a neighbor graph representation of the system (Fig. 1d). Therefore, the plugin interfaces `chemtrain-deploy` with framework-specific algorithms for parallelization and to run advanced MD simulations with the exported model. In the current version, `chemtrain-deploy` provides a plugin to the MD software LAMMPS [29].

Finally, `chemtrain-deploy` provides a library for the plugin to evaluate the exported model (Fig. 1e). This library uses XLA [54] and PJRT [55] to translate the model into an efficient backend-specific computation at runtime. Therefore, the library

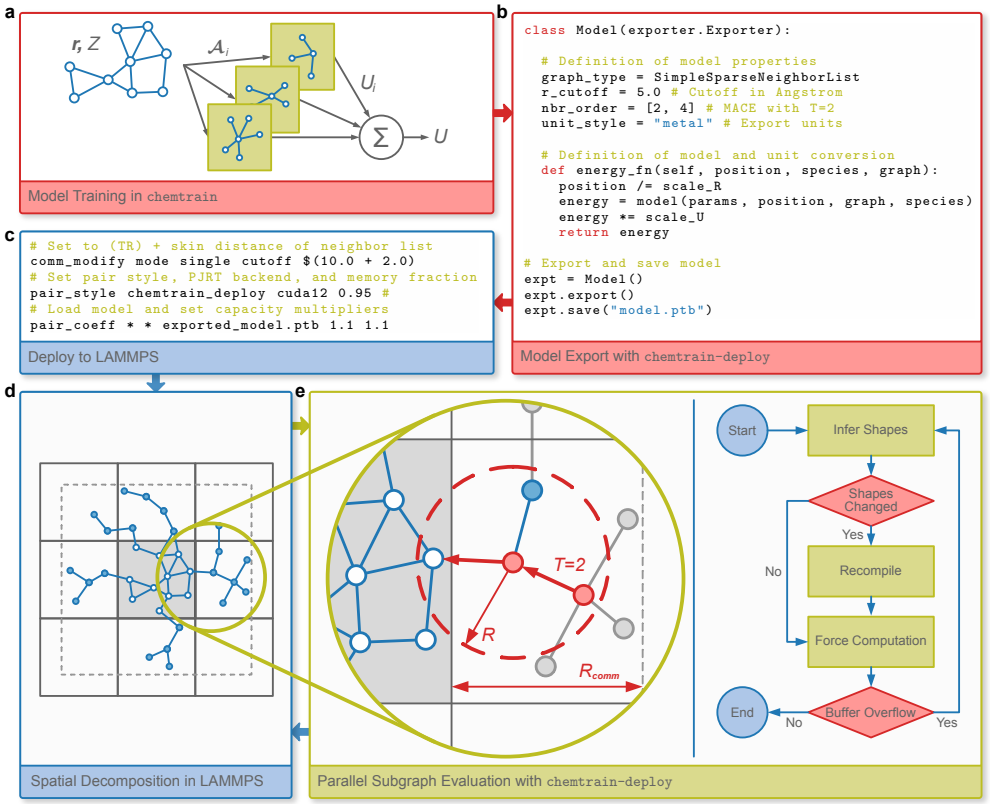


Fig. 1 Overview of `chemtrain-deploy`. Model trained in `chemtrain` **(a)** is exported **(b)** and loaded into LAMMPS **(c)**. LAMMPS distributes the workload onto multiple processors by decomposing the system into domains (solid lines) and computes domain neighbor lists including atoms in the domain (blue empty) and atoms from other domains (blue filled) within $R_{\text{comm}} \geq TR$ (dashed gray lines) **(d)**. `chemtrain-deploy` computes the potential and forces independently for each domain, pruning atoms (gray) from the neighbor list graph generated by LAMMPS and buffering atom and graph data to fixed shapes required by XLA **(e)**.

transforms and buffers the MD softwares's atom and neighbor data for compilation with XLA. Following, the XLA compiler performs hardware-independent optimizations such as common subexpression elimination and operation fusion to reduce computational cost and memory requirements. Finally, the pluggable PJRT runtimes further optimize the code for specific backends, such as GPUs and CPUs, considering the backend's architecture. Thus, the library extends the LAMMPS' capabilities to run efficiently on specific hardware and evaluate new force-field architectures without rewriting or recompiling LAMMPS. Moreover, the shared library promotes future extension of `chemtrain-deploy` by reusing provided functionality in new plugins, respectively extensions, to other MD software.

Distributed potential computation

In the following, we describe in more detail how `chemtrain-deploy` approaches distributed potential and force computation for semi-local potentials (see section 4.1). For these potentials, we expect that the total potential energy of an N atom system

$$U(\boldsymbol{r}) = \sum_{i=1}^{N} U_i\left(\boldsymbol{r}_i, \mathcal{A}_i\right) \tag{1}$$

decomposes into a sum of semi-local per-atom energies U_i that depend on the position of the atom $\boldsymbol{r}_i$ and the atoms $\mathcal{A}_i$ within the semi-local environment of atom i. Given a graph of the system, which represents atoms by nodes that share edges if closer to each other than a cutoff distance R, the local environment $\mathcal{A}_i = \{(\boldsymbol{r}_j, Z_j) \mid j \in \mathcal{N}_{\leq T}(i)\}$ contains positions $\boldsymbol{r}_j$ and species Z_j of all atoms that are direct neighbors $\mathcal{N}_{=1}(i)$ of node i. For semi-local potential models such as message-passing GNNs with T message-passing steps, the local environment additionally contains atoms i, referred to as T-th order neighbors $\mathcal{N}_{\leq T}(i)$, to which a path of at most length T exists (see Fig. 1e).

`chemtrain-deploy` computes the potential energies in parallel on multiple independent processors (GPUs/CPUs) by partitioning the full graph into one subgraph per processor using spatial system decomposition and neighbor list generations provided in MD software, e.g., LAMMPS [29]. As outlined in Figure 1d, LAMMPS distributes the workload by dividing the system into non-overlapping domains, such that each atom is local to exactly one domain. LAMMPS then assigns the domains and the contained local atoms to the available processors. For each processor, LAMMPS additionally copies all atoms from other domains within a distance of R_{comm} of the processor's domain boundary and constructs a neighbor list graph. Since the maximum distance between an atom i and any of its T-th neighbors can be TR, choosing $R_{\text{comm}} \geq TR$ ensures that the neighbor graph of each domain contains all T-th neighbor atoms of all local atoms. Thus, summing up the predicted energies of local atoms results in the total potential energy of the system, as each atom energy U_i is computed exactly once on a subgraph with a complete environment of i.

Running MD simulations requires `chemtrain-deploy` to compute the forces acting on the atoms. From the sum rule, the total force on an atom

$$\boldsymbol{f}_i = -\frac{\partial U_i(\boldsymbol{r}_i, \mathcal{R}_i)}{\partial \boldsymbol{r}_i} - \sum_{j \in \mathcal{N}^R_{\leq T}} \frac{\partial U_j(\boldsymbol{r}_j, \mathcal{R}_j)}{\partial \boldsymbol{r}_i} \tag{2}$$

decomposes into a sum of partial forces $\boldsymbol{f}_{ij} = -\frac{\partial U_j(\boldsymbol{r}_j, \mathcal{R}_j)}{\partial \boldsymbol{r}_i}$. The total forces of all local atoms can be computed directly on each processor by computing all nonzero partial forces. However, this approach generally requires extending the domain subgraph to include the $2T$-th order neighbors, which are necessary to correctly compute the potential energy of all T-th order neighbors of the local atoms. Alternatively, `chemtrain-deploy` computes the partial forces of all local atoms with respect to all atoms of the domain subgraph. Then, the total forces on each local atom can be

obtained by summing up all partial forces from corresponding copies on other processors. Thus, by constructing graphs containing all T-th order neighbors of local atoms, all particles' forces and potential energies can be computed with initial and final but without intermediate communication operations.

The graph obtained from LAMMPS might not be minimal and may contain copied atoms that are not within the semi-local environment of any local atom (gray nodes and edges in Fig. 1e). Moreover, neighbor lists are typically constructed with edges longer than the model cutoff to prevent a costly neighborlist recomputation at every timestep. Therefore, `chemtrain-deploy` prunes the neighbor list graph at every timestep to improve the costly model evaluation. First, `chemtrain-deploy` removes all edges from the graph that are longer than the specified cutoff. Following `chemtrain-deploy` identifies all T-th neighbors of the local atoms by sending out pseudo-messages from the local atoms. Each atom that received messages in the previous steps sends a message in the next step. Finally, after the T message passing steps, all T-th neighbors of local atoms have received a message. `chemtrain-deploy` automatically adds the pruning computation to the model during the export (Fig. 1b). Therefore, pruning operations are parallelized and optimized through XLA.

Parallelization cost

We estimate the cost of parallelizing semi-local potential models in homogeneous systems. In homogeneous systems, the cost of a semi-local potential model scales linearly with the number of atoms in the domain [18]. We assume that the system domains are rectangular boxes with side lengths $L_x = L_y = L_z = L$ for bulk systems periodic in all dimensions and $L_x = L_y = L \gg L_z$ for surface systems periodic in the x and y dimension, the total number of atoms is N is proportional to L^d, where d is the number of periodic dimensions. However, due to copied particles within a distance of TR to the domain boundary, the cost of computing energies and forces is proportional to $(L + 2TR)^d$.

The workload can be divided among P processors by using the domain decomposition described before to accelerate the computation. Therefore, each processor computes forces and energies for a domain of approximately length $P^{-1/d}L$ in the periodic dimension, still requiring copies of atoms within TR distance to the domain boundary. Under the assumption that the runtime is proportional to the cost, the parallelization speeds up the computation by a factor

$$S = \left(\frac{L + 2TR}{P^{-1/d}L + 2TR} \right)^d . \tag{3}$$

Since P processors have to spend a relatively higher amount of work on computing interactions between copied atoms than between local atoms, the total work increases, causing a decrease in the parallel efficiency

$$\varepsilon = \frac{S}{P}. \tag{4}$$

Runtime optimizations and buffering

`chemtrain-deploy` optimizes and evaluates the model through XLA. However, XLA re-optimizes the program every time the shape of an input changes, which typically requires more time than the actual computation. Thus, `chemtrain-deploy` buffers all dynamically shaped inputs to a fixed shape and evaluates the model as outlined in Figure 1. First, `chemtrain-deploy` computes the required buffer shapes. These shapes can vary, for example, if the number of atoms and neighbors in the domain changes. If the buffer capacities are exceeded, `chemtrain-deploy` enlarges the buffers and recompiles the model using the TensorFlow [56] call module loader. If no buffer overflowed, `chemtrain-deploy` transforms and copies data to the device and performs the computation. The models use internal buffers to enable optimizations such as graph pruning in the computation. Thus, after the computation, `chemtrain-deploy` checks whether internal model buffers overflowed. If internal model buffers overflow, `chemtrain-deploy` repeats the computation with resized model buffers. If no buffer overflowed, `chemtrain-deploy` copies back the computed forces and returns statistics of the computation.

Since `chemtrain-deploy` only enlarges buffers, the frequency of recompilations decreases for systems at equilibrium. However, recompilations can happen frequently in the initial stages of computations on multiple devices if each device recompiles independently. Thus, `chemtrain-deploy` enforces collective recompilations of multiple devices per time step by explicitly controlling recompilations. Therefore, the `chemtrain-deploy` plugin first tries to evaluate the model with recompilation disabled. If a recompilation is necessary on one device, the device raises an exception that will be called by the plugin. The plugin then synchronizes the error to all devices, which will enlarge overflown and nearly filled buffers and recompile the program. Thereby, `chemtrain-deploy` boosts simultaneous recompilations on multiple devices to shorten warm-up periods in large-scale parallel applications.

Traning state-of-the-art neural MLPs with `chemtrain`

With the flexibility of `chemtrain` in training state-of-the-art MLPs, we used it to train models on three chemically and structurally diverse systems commonly studied in biophysics and materials science: a liquid-vapor water system, a crystalline aluminum solid, and the mini-protein Chignolin solvated in water. These systems span homogeneous and heterogeneous environments and include different phase states, such as liquid, solid, and interfacial configurations, capturing a broad range of chemical and structural complexity. For each system, we chose a corresponding training dataset: H2O-PBE0TS [13], ANI-AL [31], and SPICE [57], respectively. We used three different GNN architectures that reflect the methodological diversity of modern approaches: Allegro [18], MACE [22], and PaiNN [21], on the same dataset for each system. For fair comparison, we carefully selected hyperparameters to achieve similar energy and force accuracy across architectures. In all cases, the models reached comparable MAE or RMSE values, remained within chemical accuracy, and matched reported literature benchmarks (Table 1). Further details on the datasets, training procedures, and hyperparameter configurations are provided in the Methods section.

Table 1 Root mean square (RMSE) and mean absolute errors (MAE) for energies (meV/atom) and forces (meV/Å) across Allegro, MACE, and PaiNN models on the ANI-AL, SPICE, and H_2O-PBE0TS datasets, with reference values from the literature, including classical MEAM and other MLPs.

	Allegro	**MACE**	**PaiNN**	**Reference**
ANI-AL				ANI-AL [31], MEAM [58]
Energy (RMSE)	12.9	9.9	7.2	1.9, 60.6
Force (RMSE)	109.4	71.8	62.7	60.0, 244.8
SPICE				TorchMD-NET [57]
Energy (MAE)	12.4	46.1	27.4	48.3
Forces (MAE)	73.5	47.6	48.4	–
H_2O-PBE0TS				NequIP [19], DeepMD [13]
Energy (RMSE)	0.7	0.5	0.7	0.6, 0.3
Force (RMSE)	36.2	13.3	17.2	11.6, 40.4

Memory requirements

GNN-based MLPs can require significant memory to store high-dimensional node features and messages. However, model-agnostic software such as JAX, M.D., GROMACS-NNPot, or OpenMM-Torch does not support multi-GPU simulations through domain decomposition. Therefore, we determined the maximally supported system sizes and runtimes for MD simulations using GNN potentials that can be run on a single GPU with JAX, M.D. Additionally, we report reference measurements for `chemtrain-deploy`, which, unlike JAX, M.D., is not limited to only one GPU.

We tested all combinations of models and systems, for which we report accuracies in the previous section. To increase the system sizes, we replicated all systems equally in all periodic dimensions as described in Section 4.3. For JAX, M.D., the maximum system sizes were lower than for `chemtrain-deploy` (Table 2), limited to less than half a million atoms. In comparison, `chemtrain-deploy` could simulate more or a similar number of atoms than JAX, M.D. with the strictly local Allegro model on one GPU. Differently, for the message-passing models, the maximum system sizes that could be simulated with `chemtrain-deploy` were lower than for JAX, M.D. In all cases, the runtime `chemtrain-deploy` was similar or slower than for JAX, M.D. (Supplementary Table 1).

The difference in memory consumption and computational efficiency is likely due to JAX, M.D. updating the neighbor list entirely on the GPU and applying periodic boundary conditions without copied atoms. For large systems and short-ranged models, the neighbor list generation can require more memory in JAX, M.D. than computing interactions for copied atoms in `chemtrain-deploy`. For smaller systems and models with larger effective cutoffs, memory and compute requirements for copied atoms are higher than for local atoms, affecting the computational efficiency (see Supplementary Figure 1). However, due to the copied atoms, `chemtrain-deploy` can parallelize the simulation on multiple GPUs. Therefore, using additional GPUs could compensate for the higher memory requirements. In contrast, JAX, M.D does not support parallelization, such that the memory requirements limit the maximum system

sizes below the order of a million atoms. However, applications such as the investigation of solidification can still exhibit finite-size effects up to two million atoms [59]. Thus, the single-GPU support prevents software such as JAX, M.D. from deploying GNN potentials to applications requiring large-scale simulations.

Table 2 Maximum system sizes in number of atoms for JAX, M.D. vs `chemtrain-deploy` on a single GPU (A100, 80GB) for Allegro, MACE, and PaiNN applied to solid state aluminium (fcc) at 1000 K, replicated box of solvated Chignolin at ambient conditions, and water slab at ambient conditions.

	System	JAX, M.D.	`chemtrain-deploy`
Allegro	Aluminium	296,352	470,596
	Chignolin	27,936	27,936
	Water	253,125	496,125
MACE	Aluminium	202,612	108,000
	Chignolin	94,284	94,284
	Water	162,000	112,500
PaiNN	Aluminium	340,736	171,500
	Chignolin	27,936	3,492
	Water	72,000	40,500

Scaling to million-atom systems

To estimate the performance of `chemtrain-deploy` for simulating large systems on multiple GPUs, we evaluated strong and weak scaling for all combinations of systems and models. For each combination, we selected a different system size to respect the different memory requirements of the models. For the strictly local Allegro model, we observed close-to-ideal strong scaling (Figure 2), slightly outperforming the anticipated strong scaling in Eq. 3 and often exceeding the anticipated ideal parallel efficiency (Supplementary Figure 2). This improvement might be due to XLA optimizations leveraging additional memory and compute resources. For the message-passing GNNs, we obtained good strong scaling, except for Chignolin simulated with the PaiNN model. In all cases, the measured strong scaling is consistent with our approximation given in Eq. 3 for all systems.

As shown in Figure 3, all models exhibited close-to-ideal weak scaling. This result indicates that inter-device and inter-node communications do not crucially affect efficiency for multi-GPU computations. Therefore, scaling in `chemtrain-deploy` is mostly determined by the effort spent on copied atoms compared to local atoms (Supplementary Figure 1). Thus, Eq. 3 provides a good reference to estimate the required cost and resources of scaling message-passing GNNs to large systems.

To directly compare the models, we also evaluated the scaling on million-atom systems, visualized in Figure 4 (simulation speeds reported in Supplementary Table 2). The Allegro and MACE models showed good strong scaling for all systems. The PaiNN

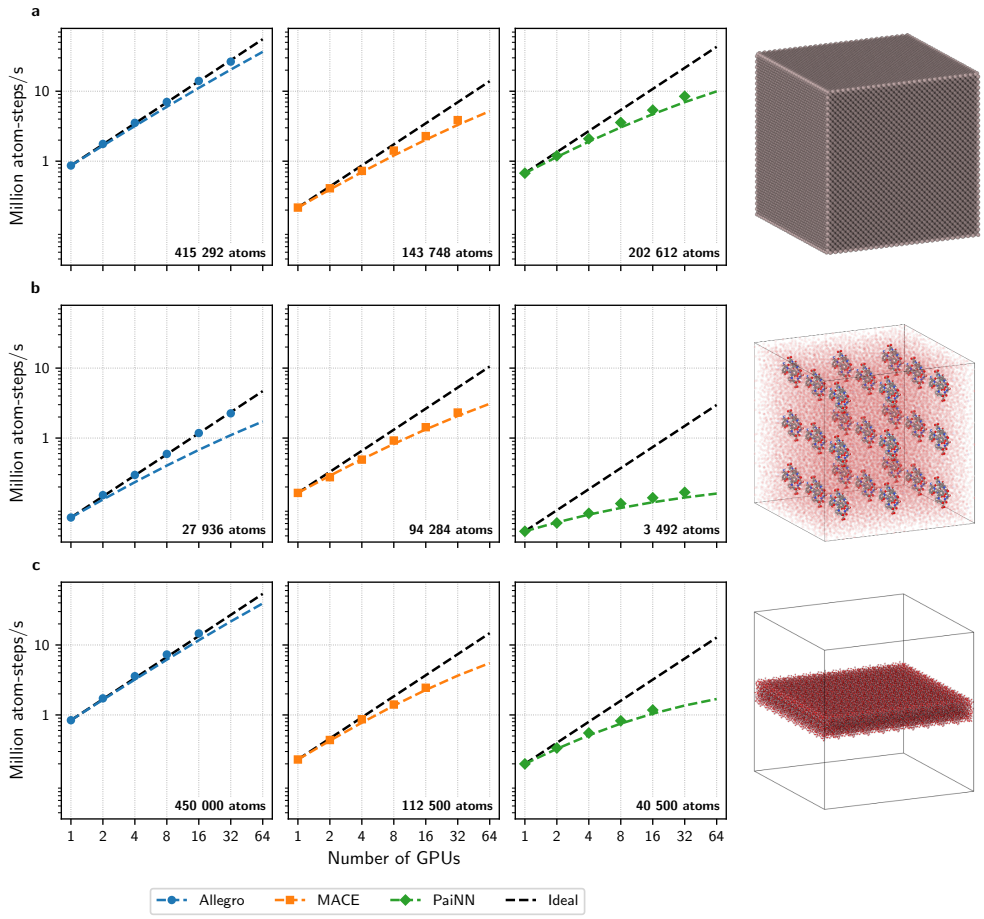


Fig. 2 Strong scaling on JEDI for Allegro (blue), MACE (orange), and PaiNN (green) applied to **a** solid state aluminium (fcc) at 1000 K, **b** replicated box of solvated Chignolin at ambient conditions, and **c** water slab at ambient conditions next to visualizations of the systems. The sizes of each system are given in numbers of atoms in the lower right corner. Visualizations were created with OVITO [60] for the systems corresponding to the MACE model. Ideal and approximate (Eq. 3) strong scaling are shown as dashed lines in black and the model colors, respectively.

model scaled slightly worse than the other models in the aluminium system and failed for the Chignolin and water systems due to insufficient memory. Comparing the simulation speed of all models, the Allegro model performed best for aluminium and water. The MACE models achieved a similar throughput for all systems, outperforming the speed of Allegro for the Chignolin system. The PaiNN model performed similarly to the Allegro model in the aluminium system. Based on these scaling results, we conclude that the Allegro model is highly efficient for systems with a few atom types, such as water and aluminium. However, for chemically diverse systems, MACE provides a better tradeoff between accuracy, scalability, and robustness (Supplementary

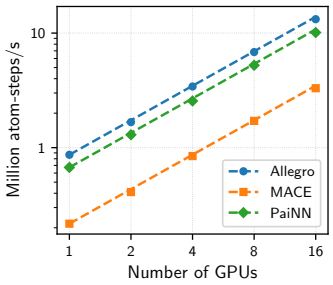


Fig. 3 Weak scaling on JEDI for Allegro (blue), MACE (orange), and PaiNN (green) applied to solid state aluminium (fcc) systems at 1000 K with 415, 292, 143, 748, and 202, 612 atoms per GPU. Ideal weak scaling is displayed as dashed lines for each model in the respective color.

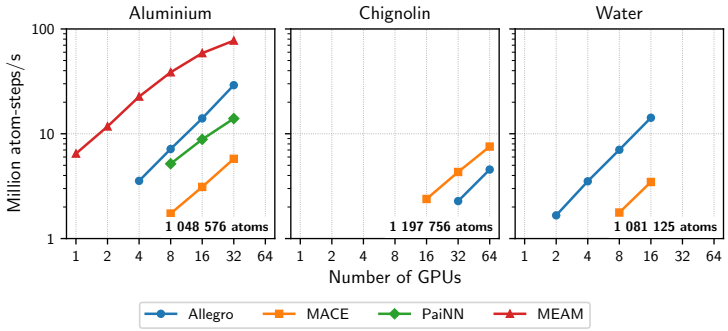


Fig. 4 Strong scaling on JEDI for Allegro (blue markers and lines), MACE (orange markers and lines), and PaiNN (green markers and lines) applied to solid state aluminium (fcc) at 1000 K (left), replicated box of solvated Chignolin at ambient conditions (middle), and water slab at ambient conditions (right) for systems with approximately 1 million atoms. The exact numbers of atoms are shown in bold in the lower right corners of the plots. Missing results correspond to simulations that failed due to insufficient memory. Scaling for the modified embedded atom method (MEAM) potential [58] applied to the aluminium system is shown as reference (red markers and line).

Note 1). Using PaiNN with a larger effective cutoff can be beneficial for very simple systems, but requires extensive memory for chemically more diverse systems.

We additionally compare the scaling with other implementations and models as a reference. For the aluminium system, all models scaled better than a classical MEAM potential [58] (Figure 4) but were slower for the same number of GPUs. The computational speed of the MEAM potential is naturally higher due to a simpler computation, which affects the model's accuracy (Table 1). The Allegro models for the aluminium and water system showed strong scaling comparable to Allegro models deployed to similar systems with Allegro-LAMMPS [18, 61] (Supplementary Figure 3). However, the exact difference in computational speed depends on the model architecture, such as the depth of the model and the cutoff of the graph.

3 Discussion

We present `chemtrain-deploy`, a model-agnostic framework for deploying JAX-based semi-local MLPs to LAMMPS. By coupling JAX-based models with the scalability and functionality of LAMMPS, `chemtrain-deploy` provides a seamless interface for running complex and large-scale simulations with minimal integration overhead on multiple GPUs. Therefore, `chemtrain-deploy` overcomes key limitations of existing software, which are often restricted to specific model architectures, provide limited training support, or face scalability challenges.

Our results demonstrate excellent scaling of modern GNN potentials through `chemtrain-deploy` for different systems, particularly in simulations involving millions of atoms across multiple GPUs. Through optimizations such as XLA-based compilation and graph pruning, we reduce execution overhead and minimize recompilation costs. This capability enables the application of semi-local MLPs to new fields in computational biology and material sciences.

We compared different state-of-the-art GNN architectures. We found that strictly local models generally exhibit superior scalability due to their limited communication overhead, while semi-local message-passing GNNs tend to provide improved accuracy but can exhibit reduced scalability and increased memory demands. Nonetheless, actual computational performance depends on the specific MLP hyperparameters and the systems of interest to practitioners. These insights might provide a starting point for future users to select architectures and guide the development of MLPs.

`chemtrain-deploy` can also support other semi-local MLPs beyond those models demonstrated in this paper, such as Behler-Parinello potentials [62], NequIP [19], DimeNet++ [63], and future MLPs likely to be available in a JAX-based implementation. Moreover, these architectures can be combined, e.g., with classical field priors for coarse-grained systems [33, 64], or Coulomb interactions and dispersion corrections [65] to form hybrid classical/GNN potentials with high stability and effective treatment of long-range interactions [36–40]. From a future perspective, the flexibility and extensibility of `chemtrain-deploy` ensure its long-term usability beyond current state-of-the-art models and foster innovation through enabling rapid implementation and testing.

Looking forward, several promising avenues exist to extend the capabilities of `chemtrain-deploy`. These include support for global models that can capture long-range interactions [66], integration with other popular MD software such as GROMACS [28] and NAMD [67], implementation of adaptive cutoff schemes [61], and multiscale modeling techniques such as multi-time-step algorithms [68]. Such developments will expand the applicability of `chemtrain-deploy` to an expanded set of complex systems and simulation scenarios, further bridging the gap between MLPs and practical, large-scale molecular simulations.

The broad applicability of `chemtrain-deploy` opens several exciting opportunities for MD community. Its ability to efficiently handle million-atom systems enables simulations of complex materials and biological systems. By making use of the various functions of LAMMPS, `chemtrain-deploy` supports non-standard simulation protocols such as those under external fields or using enhanced sampling techniques, expanding the range of molecular phenomena that can be studied. Importantly,

its model-agnostic design allows easy benchmarking and comparison of different machine learning potentials within consistent simulation settings, accelerating the development and validation of next-generation models. Together, we expect that `chemtrain-deploy` will accelerate the adoption and development of advanced machine learning potentials, enabling transformative advances in large-scale molecular simulations and ultimately driving progress across computational chemistry and materials science.

4 Methods

4.1 (Semi-)Local Potential Models

Molecular dynamics simulations can describe the behavior of atomistic systems through forces $\boldsymbol{f}_i$ that derive from a potential energy function $\boldsymbol{f}_i = \frac{\partial U(\boldsymbol{r})}{\partial \boldsymbol{r}_i}$. In many systems, atoms predominantly interact with other atoms in their local environment. Therefore, many classical and modern models approximate the total energy of a system through a sum of local atomic energy contributions

$$U(\boldsymbol{r}) = \sum_{i=1}^{N} U_i\left(\boldsymbol{r}_i, \mathcal{A}_i\right), \tag{5}$$

where $\mathcal{A} = \{(\boldsymbol{r}_j, Z_j) \mid j \in \mathcal{N}_{=1}(i)\}$ is the set of atom positions and species Z_i of the direct neighbors $\mathcal{N}_{=1}(i)$ to particle i with a distance $\|\boldsymbol{r}_{ij}\|$ less than a cutoff. [69, 70]

Descriptors

The predicted potential energy should be invariant to permutations of atoms of the same species and translation, rotation, and reflections of the reference frame [69, 70]. However, applying general regression models such as Neural Networks or Kernel Models directly to the atomic positions does not necessarily result in a model that respects these invariances [69]. Therefore, many potential models first encode the atomic environments into an invariant vector of local descriptors $\boldsymbol{\xi} = \{\xi^\alpha\left(\boldsymbol{r}_i, \mathcal{A}_i\right)\}$ that serve as input to an learnable atomic energy function $U_i(\boldsymbol{r}, \mathcal{A}_i) = \tilde{U}_i\left(\boldsymbol{\xi}_i\left(\boldsymbol{r}_i, \mathcal{A}_i\right)\right)$ [70] such as a Neural Network [69]. Using the same model and descriptors for all particles of the same species ensures geometric and permutation invariant potential predictions [69].

Local descriptors typically achieve translational invariance by acting on the atom displacements $\boldsymbol{r}_{ij}$ rather than the absolute positions [70]. Moreover, descriptors such as the Atom-Centered Symmetry Functions (ACSF) achieve rotation and reflection invariance by encoding distances and directions between pairs and triplets of neighbors through invariant distances and angles. However, this encoding scales unfavorably with the number of neighbors for more than two-body correlations. Therefore, equivariant descriptors such as the Smooth Overlap of Atomic Positions (SOAP) [71] represent displacements in a suitable basis and perform equivariant operations to encode correlations between multiple neighbors while scaling linearly with the number of neighbors. More generally, the Atomic Cluster Expansion (ACE) provides a systematic approach to constructing a complete basis for the local environment through a hierarchical

expansion. Thus, the ACE descriptor can represent many previously proposed local descriptors while scaling linearly with the number of neighbors [12].

Graph Neural Networks

Devising accurate and efficient descriptors by hand for chemically diverse datasets is difficult. Approaches such as ACE describe how to systematically build a complete basis for the local environment [70]. However, the resulting descriptors scale unfavorably with the number of atom species due to the number of basis functions [18]. Thus, learning efficient environment descriptors from data through Graph Neural Networks (GNNs) has gained wide attention.

GNNs encode locality by representing the system as a graph, with nodes representing atoms and edges connecting all neighbors $\mathcal{N}_{=1}$. The graph is commonly embedded by assigning node features $h_i^{(0)} = f_h(Z_i)$ based only on the particle species to ensure permutation invariance, and edges features $e_{ij}^{(0)} = f_e(\boldsymbol{r}_{ij})$ based on edge displacements to ensure translational invariance. Many proposed models, such as SchNet [20] or DimeNet [72], then extract environment descriptors through the Message-Passing (MP) framework [11] by propagating information along the graph through messages

$$\boldsymbol{m}_i^{t+1} = \sum_{j \in \mathcal{N}(i)} \mathcal{M}^t(\boldsymbol{h}_i^t, \boldsymbol{h}_j^t, \boldsymbol{e}_{ij}), \tag{6}$$

which aggregate information from neighboring atoms by a learnable message function $\mathcal{M}^t$. Using the aggregated messages, the MP-GNNs then update the node features

$$\boldsymbol{h}_i^{t+1} = \mathcal{U}^t(\boldsymbol{m}_i^{t+1}, \boldsymbol{h}_i^t), \tag{7}$$

through an update function $\mathcal{U}^t$. The final node features $\boldsymbol{h}_i^L$ after T message passing steps can act as input to a regression model such as a Neural Network [20] or a Gaussian Process [73].

Similar to classical descriptors, the GNN predictions must be invariant to translations, rotations, and reflections. Early GNNs achieved this invariance through an invariant graph embedding using distances [20] and angles [72]. However, higher-order embeddings can improve the GNN expressiveness but scale unfavorably with the number of neighbors [18], similar to ACSFs. Moreover, propagated messages contain only invariant information about the particle's local environment, prohibiting leveraging information about their relative orientation [21]. Therefore, equivariant MP-GNNs generalize message-passing to tensorial features, such as displacements between atoms, to efficiently propagate directional information about atomic environments. Thereby, equivariant GNNs employ operations that ensure tensorial features are equivariant, i.e., transform similarly to the input for a group of transformations, to ensure invariance of the final scalar node features [17, 19, 21].

Unlike the previously described strictly local descriptors, messages passing GNNs can propagate information between atoms that are not direct neighbors. Instead, GNNs propagate information from an atom i to higher-order neighbor atoms $\mathcal{N}_{\leq T}$ connected by a path of length less or equal than T. Thus, MP-GNNs with many

message-passing layers have a large receptive field of radius TR, potentially impairing their scalability. To ensure high scalability, multiple approaches aim at constructing descriptive GNNs with small receptive fields. The Multi-ACE framework [17] reformulates message construction in the ACE formalism, generalizing previous invariant and equivariant MP-GNNs, such as SchNet [20], DimeNet [72], NequIP [19], and PaiNN [21], that correlate only information from a limited number of neighbors for each message. Through the ACE formalism, this framework enables models such as MACE [22] to construct messages that correlate information from an arbitrary number of neighbors to exploit high-order many-body correlations independently of the number of message-passing steps while scaling linearly with the number of neighbors. The Allegro model [18] reformulates message-passing in an edge-centric formalism. Therefore, Allegro only passes messages between directed edges originating from the same node. Consequently, no information is propagated to particles outside the cutoff shell. Consequently, the Allegro model learns strictly local environment descriptions.

Chosen Architectures

In this work, we chose the GNN models PaiNN, MACE, and Allegro as examples of different design choices of models in terms of receptive fields and fidelity of semi-local descriptions. On the one hand, the PaiNN model only correlates pairs of neighbor features in each message-passing layer, such that the number of message-passing layers determines the receptive field and the body order of the final descriptor. On the other hand, the Allegro model employs equivariant edge-based message passing to ensure strict locality, such that the number of message-passing layers only affects the body order but not the receptive field of the model. The MACE model employs the ACE formalism to correlate information from a variable number of neighbors. Therefore, the final body order can be enlarged without increasing the receptive field.

4.2 Reference Training Datasets

ANI-AL

The dataset includes over 6000 DFT calculations on supercells containing up to 250 atoms, covering a wide range of nonequilibrium configurations. It was generated using a minimally guided active learning approach. The data can be obtained from https://github.com/atomistic-ml/ani-al.

SPICE

SPICE is a quantum chemistry dataset for simulating drug-like small molecules and proteins. It contains over 1.1 million configurations, representing a diverse set of small molecules, dimers, dipeptides, and solvated amino acids. The dataset provides energies and forces computed using the ωB97M-D3(BJ)/def2-TZVPPD level of theory. The data can be obtained from https://github.com/openmm/spice-dataset.

H_2O-PBE0TS

The H_2O-PBE0TS dataset contains snapshots of liquid water and ice configurations generated via ab initio molecular dynamics (AIMD) using the PBE0+TS

functional. The data can be obtained at https://aissquare.com/datasets/detail?pageType=datasets&name=H2O-PBE0TS.

4.3 Molecular Dynamics Simulations

MD simulations were performed using LAMMPS with the `chemtrain-deploy` pair style implemented in our custom `chemtrain-deploy` interface. All simulations, including production and timing runs, were carried out in the NVT ensemble using a Nosé–Hoover thermostat with a temperature damping parameter of 1.1 ps. Each system underwent 100 equilibration steps followed by 250 production steps. For the aluminium case, a face-centered cubic lattice (lattice constant 4.065 Å) was constructed and replicated equally in all three dimensions; simulations were conducted at 1000 K with a 3 fs time step and a 2.0 Å neighbor skin. For the Chignolin case, the system was read from a preconfigured structure created by GROMACS, consisting of a Chignolin molecule solvated in a 3.3 nm cubic TIP3P water box. The system was then replicated equally in all three spatial dimensions according to the scaling index. Simulations were performed at 293.15 K using a 0.5 ps time step and a 2.5 Å neighbor skin distance. For the water–vapor interface case, the system was initialized from a pre-equilibrated 2×2×5 nm^3 TIP3P water box and replicated equally in the x and y directions according to a scaling index; the z-direction was extended to create vacuum regions for a water–vacuum interface, similar to prior setup [74]. This simulation was run at 293.15 K using a 1 fs time step and a 2.5 Å neighbor skin distance. Simulations performed using JAX, M.D. followed the same settings and initial configurations as the corresponding LAMMPS simulations.

Scaling simulations were conducted on the JEDI test system using up to 16 nodes interconnected via InfiniBand NDR200. Each node consists of 4 NVIDIA GH200 Superchips, with each Superchip pairing 72 CPU cores and an H100 GPU with 96GB of memory. JAX-M.D. simulations were performed on a single A100 GPU with 80GB of memory.

4.4 Training Details

The following training settings are kept consistent across all models, with architecture-specific hyperparameters detailed in the corresponding subsections. All models are trained using a force-matching approach. Given a dataset of atomic configurations with reference energies U^{ref} and reference forces $\boldsymbol{f}^{\mathrm{ref}}$, we optimize the neural network parameters θ to minimize differences between predicted and reference values. The training loss is defined as:

$$\mathcal{L}(\theta) = \lambda_E \sum_{\alpha} \left| U^{\theta}(\boldsymbol{r}_\alpha) - U^{\mathrm{ref}}(\boldsymbol{r}_\alpha) \right|^2 + \lambda_F \sum_{\alpha,i} \left\| \boldsymbol{f}_i^{\theta}(\boldsymbol{r}_\alpha) - \boldsymbol{f}_i^{\mathrm{ref}}(\boldsymbol{r}_\alpha) \right\|^2, \tag{8}$$

where λ_E and λ_F control the relative weighting of the energy and force terms. All models are trained using single-precision (FP32) arithmetic and the Adam optimizer with default parameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$. For each dataset, a consistent train–test–validation split ratio of 7:2:1 is used, where we used the validation

split to select the best performing parameters. For MACE, the energy and force weights in the loss function are set to $\lambda_U = 10^{-6}$ and $\lambda_F = 10^{-1}$, respectively. For PaiNN, we use $\lambda_U = 10^{-4}$ and $\lambda_F = 10^{-1}$. For Allegro on water and aluminum, the weights are $\lambda_U = 10^{-6}$ and $\lambda_F = 10^{-1}$, while for Chignolin, both are set to $\lambda_U = 10^{-4}$ and $\lambda_F = 10^{-4}$.

All models use a graph cutoff distance of 5 Å. For all Allegro models on water and alumnium, we use one tensor product layer with $l_{\max} = 3$, 8 radial basis functions, and a polynomial envelope of order 2, while we use three layers with $l_{\max} = 2$ for Chignolin. For MACE models, we set the hidden irreducible representations to "$32 \times 0e + 32 \times 1o$" across all cases, with $l_{\max} = 3$, a correlation order of 3 per layer, 2 interaction layers, a node embedding dimension of 64, 8 radial basis functions, and a polynomial envelope of order 6. For PaiNN, we use 4 layers in all cases and vary only the size of the embedding features, keeping all other hyperparameters fixed. The learning rate generally follows a polynomial decay schedule with a power of 2.0 and a decay rate 10^{-5}. The only exception is for the Allegro model on the SPICE dataset, where we use an exponential decay schedule with a decay rate of 0.001.

All models were trained on a single NVIDIA A100 GPU.

H_2O-PBE0TS and ANI-AL Models

The H_2O-PBE0TS models were trained on a total of 100,000 samples. For Allegro, we use a hidden MLP layer dimension of 64 and embedding dimensions of [8, 16, 32]. The hidden irreducible representations are set to "$32 \times 0e + 16 \times 1e + 16 \times 1o + 8 \times 2e + 8 \times 2o$". For both MACE and Allegro, the learning rate is set to 0.01. For PaiNN, we use a hidden feature size of 128 with a initial learning rate of 0.001.

The ANI-AL models were trained on 6,000 samples. All hyperparameters are kept the same as in the H_2O-PBE0TS case, except that the PaiNN hidden feature size is set to 64. The learning rates remain the same, while the batch sizes are 64 for Allegro, 16 for MACE, and 8 for PaiNN.

SPICE Models

We use the entire dataset for training, excluding the Ion Pairs subset, with a total of 1,817,199 samples. For Allegro, the hidden MLP layer dimension is set to 256, with embedding dimensions of [128, 128, 256]. The hidden irreducible representations are set to "$64 \times 1o + 16 \times 2e$". For both Allegro and MACE, the initial learning rate is 0.001, with batch sizes of 16. For PaiNN, we use a hidden feature size of 128, a learning rate of 10^{-4}, and a batch size of 32.

Acknowledgments

Funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them. This work was funded by the ERC (StG SupraModel) - 101077842 and the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) - 534045056 and 561190767.

We gratefully acknowledge the Gauss Centre for Supercomputing e.V. (www.gauss-centre.eu) for supporting this project with computing time provided through the John von Neumann Institute for Computing (NIC) on the GCS Supercomputer JEDI at the Jülich Supercomputing Centre (JSC).

The authors thank Jan Eckwert and Ian Störmer for valuable discussions and Mario Geiger for open-sourcing his Allegro JAX code.

Data Availability

The H_2O-PBE0TS, ANI-AL and SPICE datasets are publicly accessible. (see "Methods") The parameters for the MEAM potential for aluminium can be accessed at https://www.ctcms.nist.gov/potentials/entry/2003--Lee-B-J-Shim-J-H-Baskes-M-I--Al/.

Code Availability

The `chemtrain` framework, including `chemtrain-deploy`, is open-source and available at https://github.com/tummfm/chemtrain with documentation avnailable at https://chemtrain.readthedocs.io/en/latest/. The LAMMPS molecular dyamics package is publicly available at https://github.com/lammps/lammps. Scripts for model definition, training, and benchmarking will be made publicly available at https://github.com/tummfm/chemsim-lammps upon publication of this work.

References

[1] Unke, O.T., Chmiela, S., Sauceda, H.E., Gastegger, M., Poltavsky, I., Schütt, K.T., Tkatchenko, A., Müller, K.-R.: Machine Learning Force Fields **121**(16), 10142–10186 https://doi.org/10.1021/acs.chemrev.0c01111

[2] Noé, F., Tkatchenko, A., Müller, K.-R., Clementi, C.: Machine Learning for Molecular Simulation **71**, 361–390 https://doi.org/10.1146/annurev-physchem-042018-052331

[3] Behler, J.: Perspective: Machine learning potentials for atomistic simulations. The Journal of chemical physics **145**(17) (2016)

[4] Merchant, A., Batzner, S., Schoenholz, S.S., Aykol, M., Cheon, G., Cubuk, E.D.: Scaling deep learning for materials discovery **624**(7990), 80–85 https://doi.org/10.1038/s41586-023-06735-9

[5] Iftimie, R., Minary, P., Tuckerman, M.E.: Ab initio molecular dynamics: Concepts, recent developments, and future trends **102**(19), 6654–6659 https://doi.org/10.1073/pnas.0500193102

[6] Cornell, W.D., Cieplak, P., Bayly, C.I., Gould, I.R., Merz, K.M., Ferguson, D.M., Spellmeyer, D.C., Fox, T., Caldwell, J.W., Kollman, P.A.: A Second Generation

Force Field for the Simulation of Proteins, Nucleic Acids, and Organic Molecules **117**(19), 5179–5197 https://doi.org/10.1021/ja00124a002

[7] Nikitin, A.M., Milchevskiy, Y.V., Lyubartsev, A.P.: A new AMBER-compatible force field parameter set for alkanes **20**(3), 2143 https://doi.org/10.1007/s00894-014-2143-6

[8] Marrink, S.J., Vries, A.H., Mark, A.E.: Coarse Grained Model for Semiquantitative Lipid Simulations **108**(2), 750–760 https://doi.org/10.1021/jp036508g

[9] Behler, J.: Atom-centered symmetry functions for constructing high-dimensional neural network potentials **134**(7), 074106 https://doi.org/10.1063/1.3553717

[10] Smith, J.S., Isayev, O., Roitberg, A.E.: ANI-1: An extensible neural network potential with DFT accuracy at force field computational cost. Chemical Science **8**(4), 3192–3203 (2017) https://doi.org/10.1039/C6SC05720A

[11] Gilmer, J., Schoenholz, S.S., Riley, P.F., Vinyals, O., Dahl, G.E.: Neural message passing for quantum chemistry. In: Precup, D., Teh, Y.W. (eds.) Proceedings of the 34th International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 70, pp. 1263–1272. PMLR, ??? (2017-08-06/2017-08-11)

[12] Drautz, R.: Atomic cluster expansion for accurate and transferable interatomic potentials. Physical Review B **99**(1), 014104 (2019) https://doi.org/10.1103/PhysRevB.99.014104

[13] Zhang, L., Han, J., Wang, H., Car, R., E, W.: Deep Potential Molecular Dynamics: A Scalable Model with the Accuracy of Quantum Mechanics **120**(14), 143001 https://doi.org/10.1103/PhysRevLett.120.143001

[14] Rhodes, B., Vandenhaute, S., Šimkus, V., Gin, J., Godwin, J., Duignan, T., Neumann, M.: Orb-v3: Atomistic Simulation at Scale. https://doi.org/10.48550/arXiv.2504.06231 . http://arxiv.org/abs/2504.06231

[15] Zhang, D., Peng, A., Cai, C., Li, W., Zhou, Y., Zeng, J., Guo, M., Zhang, C., Li, B., Jiang, H., Zhu, T., Jia, W., Zhang, L., Wang, H.: Graph Neural Network Model for the Era of Large Atomistic Models. https://doi.org/10.48550/arXiv.2506.01686 . http://arxiv.org/abs/2506.01686

[16] Kovács, D.P., Moore, J.H., Browning, N.J., Batatia, I., Horton, J.T., Pu, Y., Kapil, V., Witt, W.C., Magdău, I.-B., Cole, D.J., Csányi, G.: MACE-OFF: Short-Range Transferable Machine Learning Force Fields for Organic Molecules **147**(21), 17598–17611 https://doi.org/10.1021/jacs.4c07099

[17] Batatia, I., Batzner, S., Kovács, D.P., Musaelian, A., Simm, G.N.C., Drautz, R., Ortner, C., Kozinsky, B., Csányi, G.: The design space of E(3)-equivariant atom-centred interatomic potentials. Nature Machine Intelligence **7**(1), 56–67 (2025)

https://doi.org/10.1038/s42256-024-00956-x

[18] Musaelian, A., Batzner, S., Johansson, A., Sun, L., Owen, C.J., Kornbluth, M., Kozinsky, B.: Learning local equivariant representations for large-scale atomistic dynamics. Nature Communications **14**(1), 579 (2023) https://doi.org/10.1038/s41467-023-36329-y

[19] Batzner, S., Musaelian, A., Sun, L., Geiger, M., Mailoa, J.P., Kornbluth, M., Molinari, N., Smidt, T.E., Kozinsky, B.: E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. Nature Communications **13**(1), 1–11 (2022) https://doi.org/10.1038/s41467-022-29939-5

[20] Schütt, K.T., Sauceda, H.E., Kindermans, P.-J., Tkatchenko, A., Müller, K.-R.: SchNet - a deep learning architecture for molecules and materials. The Journal of Chemical Physics **148**(24), 241722 (2018) https://doi.org/10.1063/1.5019779 arXiv:1712.06113 [cond-mat, physics:physics]

[21] Schütt, K.T., Unke, O.T., Gastegger, M.: Equivariant Message Passing for the Prediction of Tensorial Properties and Molecular Spectra. https://doi.org/10.48550/arXiv.2102.03150 . http://arxiv.org/abs/2102.03150

[22] Batatia, I., Kovács, D.P., Simm, G.N.C., Ortner, C., Csányi, G.: MACE: Higher Order Equivariant Message Passing Neural Networks for Fast and Accurate Force Fields. https://doi.org/10.48550/arXiv.2206.07697 . http://arxiv.org/abs/2206.07697

[23] Schütt, K.T., Kessel, P., Gastegger, M., Nicoli, K.A., Tkatchenko, A., Müller, K.-R.: SchNetPack: A Deep Learning Toolbox For Atomistic Systems **15**(1), 448–455 https://doi.org/10.1021/acs.jctc.8b00908

[24] Gao, X., Ramezanghorbani, F., Isayev, O., Smith, J.S., Roitberg, A.E.: TorchANI: A Free and Open Source PyTorch-Based Deep Learning Implementation of the ANI Neural Network Potentials **60**(7), 3408–3415 https://doi.org/10.1021/acs.jcim.0c00451

[25] Doerr, S., Majewski, M., Pérez, A., Krämer, A., Clementi, C., Noe, F., Giorgino, T., De Fabritiis, G.: TorchMD: A Deep Learning Framework for Molecular Simulations **17**(4), 2355–2363 https://doi.org/10.1021/acs.jctc.0c01343

[26] Han, K., Deng, B., Farimani, A.B., Ceder, G.: DistMLIP: A Distributed Inference Platform for Machine Learning Interatomic Potentials. https://doi.org/10.48550/arXiv.2506.02023 . http://arxiv.org/abs/2506.02023

[27] Anderson, J.A., Lorenz, C.D., Travesset, A.: General purpose molecular dynamics simulations fully implemented on graphics processing units **227**(10), 5342–5359 https://doi.org/10.1016/j.jcp.2008.01.047

[28] Pronk, S., Páll, S., Schulz, R., Larsson, P., Bjelkmar, P., Apostolov, R., Shirts, M.R., Smith, J.C., Kasson, P.M., Van Der Spoel, D.: GROMACS 4.5: A high-throughput and highly parallel open source molecular simulation toolkit **29**(7), 845–854

[29] Thompson, A.P., Aktulga, H.M., Berger, R., Bolintineanu, D.S., Brown, W.M., Crozier, P.S., in 't Veld, P.J., Kohlmeyer, A., Moore, S.G., Nguyen, T.D., Shan, R., Stevens, M.J., Tranchida, J., Trott, C., Plimpton, S.J.: LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. Computer Physics Communications **271**, 108171 (2022) https://doi.org/10.1016/j.cpc.2021.108171

[30] Zhang, Y., Wang, H., Chen, W., Zeng, J., Zhang, L., Wang, H., E, W.: DP-GEN: A concurrent learning platform for the generation of reliable deep learning based potential energy models **253**, 107206 https://doi.org/10.1016/j.cpc.2020.107206

[31] Smith, J.S., Nebgen, B., Mathew, N., Chen, J., Lubbers, N., Burakovsky, L., Tretiak, S., Nam, H.A., Germann, T., Fensin, S., Barros, K.: Automated discovery of a robust interatomic potential for aluminum **12**(1), 1257 https://doi.org/10.1038/s41467-021-21376-0 2003.04934

[32] Levine, D.S., Shuaibi, M., Spotte-Smith, E.W.C., Taylor, M.G., Hasyim, M.R., Michel, K., Batatia, I., Csányi, G., Dzamba, M., Eastman, P., Frey, N.C., Fu, X., Gharakhanyan, V., Krishnapriyan, A.S., Rackers, J.A., Raja, S., Rizvi, A., Rosen, A.S., Ulissi, Z., Vargas, S., Zitnick, C.L., Blau, S.M., Wood, B.M.: The Open Molecules 2025 (OMol25) Dataset, Evaluations, and Models. https://doi.org/10.48550/arXiv.2505.08762 . http://arxiv.org/abs/2505.08762

[33] Thaler, S., Zavadlav, J.: Learning neural network potentials from experimental data via Differentiable Trajectory Reweighting **12**(1), 6884 https://doi.org/10.1038/s41467-021-27241-4

[34] Thaler, S., Stupp, M., Zavadlav, J.: Deep coarse-grained potentials via relative entropy minimization **157**(24), 244103 https://doi.org/10.1063/5.0124538

[35] Röcken, S., Burnet, A.F., Zavadlav, J.: Predicting solvation free energies with an implicit solvent machine learning potential **161**(23), 234101 https://doi.org/10.1063/5.0235189

[36] Cheng, B.: Latent Ewald summation for machine learning of long-range interactions **11**(1), 1–8 https://doi.org/10.1038/s41524-025-01577-7

[37] Fuchs, P., Sanocki, M., Zavadlav, J.: Learning Non-Local Molecular Interactions Via Equivariant Local Representations and Charge Equilibration. https://doi.org/10.48550/arXiv.2501.19179 . http://arxiv.org/abs/2501.19179

[38] Kosmala, A., Gasteiger, J., Gao, N., Günnemann, S.: Ewald-Based Long-Range

Message Passing for Molecular Graphs. https://doi.org/10.48550/arXiv.2303.04791 . http://arxiv.org/abs/2303.04791

[39] Caruso, A., Venturin, J., Giambagli, L., Rolando, E., Noé, F., Clementi, C.: Extending the RANGE of Graph Neural Networks: Relaying Attention Nodes for Global Encoding. https://doi.org/10.48550/arXiv.2502.13797 . http://arxiv.org/abs/2502.13797

[40] Frank, J.T., Chmiela, S., Müller, K.-R., Unke, O.T.: Euclidean Fast Attention: Machine Learning Global Atomic Representations at Linear Cost. https://doi.org/10.48550/arXiv.2412.08541 . http://arxiv.org/abs/2412.08541

[41] Fu, X., Wu, Z., Wang, W., Xie, T., Keten, S., Gomez-Bombarelli, R., Jaakkola, T.: Forces are not enough: Benchmark and critical evaluation for machine learning force fields with molecular simulations. arXiv preprint arXiv:2210.07237 (2022)

[42] Fu, X., Wood, B.M., Barroso-Luque, L., Levine, D.S., Gao, M., Dzamba, M., Zitnick, C.L.: Learning Smooth and Expressive Interatomic Potentials for Physical Property Prediction. https://doi.org/10.48550/arXiv.2502.12147 . http://arxiv.org/abs/2502.12147

[43] Póta, B., Ahlawat, P., Csányi, G., Simoncelli, M.: Thermal Conductivity Predictions with Foundation Atomistic Models. https://doi.org/10.48550/arXiv.2408.00755 . http://arxiv.org/abs/2408.00755

[44] Loew, A., Sun, D., Wang, H.-C., Botti, S., Marques, M.A.L.: Universal Machine Learning Interatomic Potentials Are Ready For Phonons. https://doi.org/10.48550/arXiv.2412.16551 . http://arxiv.org/abs/2412.16551

[45] Raja, S., Amin, I., Pedregosa, F., Krishnapriyan, A.S.: Stability-Aware Training of Machine Learning Force Fields with Differentiable Boltzmann Estimators. https://doi.org/10.48550/arXiv.2402.13984 . http://arxiv.org/abs/2402.13984

[46] Park, Y., Kim, J., Hwang, S., Han, S.: Scalable Parallel Algorithm for Graph Neural Network Interatomic Potentials in Molecular Dynamics Simulations **20**(11), 4857–4868 https://doi.org/10.1021/acs.jctc.4c00190

[47] Rohskopf, A., Sievers, C., Lubbers, N., Cusentino, M., Goff, J., Janssen, J., McCarthy, M., Zapiain, D.M.O., Nikolov, S., Sargsyan, K., Sema, D., Sikorski, E., Williams, L., Thompson, A., Wood, M.: FitSNAP: Atomistic machine learning with LAMMPS **8**(84), 5118 https://doi.org/10.21105/joss.05118

[48] Eastman, P., Galvelis, R., Peláez, R.P., Abreu, C.R.A., Farr, S.E., Gallicchio, E., Gorenko, A., Henry, M.M., Hu, F., Huang, J., Krämer, A., Michel, J., Mitchell, J.A., Pande, V.S., Rodrigues, J.P., Rodriguez-Guerra, J., Simmonett, A.C., Singh, S., Swails, J., Turner, P., Wang, Y., Zhang, I., Chodera, J.D., Fabritiis, G.D., Markland, T.E.: OpenMM 8: Molecular Dynamics Simulation

with Machine Learning Potentials. https://doi.org/10.48550/arXiv.2310.03121 . http://arxiv.org/abs/2310.03121

[49] Zeng, J., Zhang, D., Peng, A., Zhang, X., He, S., Wang, Y., Liu, X., Bi, H., Li, Y., Cai, C., Zhang, C., Du, Y., Zhu, J.-X., Mo, P., Huang, Z., Zeng, Q., Shi, S., Qin, X., Yu, Z., Luo, C., Ding, Y., Liu, Y.-P., Shi, R., Wang, Z., Bore, S.L., Chang, J., Deng, Z., Ding, Z., Han, S., Jiang, W., Ke, G., Liu, Z., Lu, D., Muraoka, K., Oliaei, H., Singh, A.K., Que, H., Xu, W., Xu, Z., Zhuang, Y.-B., Dai, J., Giese, T.J., Jia, W., Xu, B., York, D.M., Zhang, L., Wang, H.: DeePMD-kit v3: A Multiple-Backend Framework for Machine Learning Potentials https://doi.org/10.1021/acs.jctc.5c00340

[50] Zeng, J., Giese, T.J., Zhang, D., Wang, H., York, D.M.: DeePMD-GNN: A DeePMD-kit Plugin for External Graph Neural Network Potentials **65**(7), 3154–3160 https://doi.org/10.1021/acs.jcim.4c02441

[51] Fuchs, P., Thaler, S., Röcken, S., Zavadlav, J.: Chemtrain: Learning deep potential models via automatic differentiation and statistical physics. Computer Physics Communications **310**, 109512 (2025) https://doi.org/10.1016/j.cpc.2025.109512

[52] Schoenholz, S., Cubuk, E.D.: JAX MD: A Framework for Differentiable Physics **33**, 11428–11441

[53] Lattner, C., Amini, M., Bondhugula, U., Cohen, A., Davis, A., Pienaar, J., Riddle, R., Shpeisman, T., Vasilache, N., Zinenko, O.: MLIR: Scaling compiler infrastructure for domain specific computation. In: 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 2–14 (2021). https://doi.org/10.1109/CGO51591.2021.9370308

[54] OpenXLA. https://github.com/openxla/xla

[55] PJRT - Uniform Device API. https://openxla.org/xla/pjrt

[56] Developers, T.: TensorFlow. https://doi.org/10.5281/zenodo.15009305 . https://doi.org/10.5281/zenodo.15009305

[57] Eastman, P., Behara, P.K., Dotson, D.L., Galvelis, R., Herr, J.E., Horton, J.T., Mao, Y., Chodera, J.D., Pritchard, B.P., Wang, Y., De Fabritiis, G., Markland, T.E.: SPICE, A Dataset of Drug-like Molecules and Peptides for Training Machine Learning Potentials **10**(1), 11 https://doi.org/10.1038/s41597-022-01882-6

[58] Lee, B.-J., Shim, J.-H., Baskes, M.I.: Semiempirical atomic potentials for the fcc metals cu, ag, au, ni, pd, pt, al, and pb based on first and second nearest-neighbor modified embedded atom method. Phys. Rev. B **68**, 144112 (2003) https://doi.org/10.1103/PhysRevB.68.144112

[59] Mahata, A., Asle Zaeem, M.: Size effect in molecular dynamics simulation of

nucleation process during solidification of pure metals: Investigating modified embedded atom method interatomic potentials. Modelling and Simulation in Materials Science and Engineering **27**(8), 085015 (2019) https://doi.org/10.1088/1361-651X/ab4b36

[60] Stukowski, A.: Visualization and analysis of atomistic simulation data with OVITO-the open visualization tool. MODELLING AND SIMULATION IN MATERIALS SCIENCE AND ENGINEERING **18**(015012) (2010) https://doi.org/10.1088/0965-0393/18/1/015012

[61] Kozinsky, B., Musaelian, A., Johansson, A., Batzner, S.: Scaling the leading accuracy of deep equivariant models to biomolecular simulations of realistic size. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. Sc '23. Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3581784.3627041

[62] Behler, J., Parrinello, M.: Generalized Neural-Network Representation of High-Dimensional Potential-Energy Surfaces **98**(14), 146401 https://doi.org/10.1103/PhysRevLett.98.146401

[63] Gasteiger, J., Giri, S., Margraf, J.T., Günnemann, S.: Fast and Uncertainty-Aware Directional Message Passing for Non-Equilibrium Molecules. https://doi.org/10.48550/arXiv.2011.14115 . http://arxiv.org/abs/2011.14115

[64] Wang, J., Olsson, S., Wehmeyer, C., Pérez, A., Charron, N.E., De Fabritiis, G., Noé, F., Clementi, C.: Machine Learning of Coarse-Grained Molecular Dynamics Force Fields **5**(5), 755–767 https://doi.org/10.1021/acscentsci.8b00913

[65] Kabylda, A., Frank, J.T., Dou, S.S., Khabibrakhmanov, A., Sandonas, L.M., Unke, O.T., Chmiela, S., Muller, K.-R., Tkatchenko, A.: Molecular Simulations with a Pretrained Neural Network and Universal Pairwise Force Fields. https://doi.org/10.26434/chemrxiv-2024-bdfr0 . https://chemrxiv.org/engage/chemrxiv/article-details/6704263051558a15ef6478b6

[66] Fuchs, P., Sanocki, M., Zavadlav, J.: Learning Non-Local Molecular Interactions via Equivariant Local Representations and Charge Equilibration (2025)

[67] Phillips, J.C., Hardy, D.J., Maia, J.D., Stone, J.E., Ribeiro, J.V., Bernardi, R.C., Buch, R., Fiorin, G., Hénin, J., Jiang, W., et al.: Scalable molecular dynamics on cpu and gpu architectures with namd. The Journal of chemical physics **153**(4) (2020)

[68] Tuckerman, M., Berne, B.J., Martyna, G.J.: Reversible multiple time scale molecular dynamics. The Journal of Chemical Physics **97**(3), 1990–2001 (1992) https://doi.org/10.1063/1.463137

[69] Behler, J., Parrinello, M.: Generalized Neural-Network Representation of High-Dimensional Potential-Energy Surfaces. Physical Review Letters **98**(14), 146401 (2007) https://doi.org/10.1103/PhysRevLett.98.146401

[70] Musil, F., Grisafi, A., Bartók, A.P., Ortner, C., Csányi, G., Ceriotti, M.: Physics-Inspired Structural Representations for Molecules and Materials. Chemical Reviews **121**(16), 9759–9815 (2021) https://doi.org/10.1021/acs.chemrev.1c00021

[71] Bartók, A.P., Kondor, R., Csányi, G.: On representing chemical environments. Physical Review B **87**(18), 184115 (2013) https://doi.org/10.1103/PhysRevB.87.184115

[72] Gasteiger, J., Groß, J., Günnemann, S.: Directional Message Passing for Molecular Graphs. https://doi.org/10.48550/arXiv.2003.03123 . http://arxiv.org/abs/2003.03123

[73] Wollschläger, T., Gao, N., Charpentier, B., Ketata, M.A., Günnemann, S.: Uncertainty Estimation for Molecules: Desiderata and Methods (2023)

[74] Sanchez-Burgos, I., Muniz, M.C., Espinosa, J.R., Panagiotopoulos, A.Z.: A Deep Potential model for liquid–vapor equilibrium and cavitation rates of water **158**(18), 184504 https://doi.org/10.1063/5.0144500